

Operating Systems for Software Development Engineers

Complete Interview Preparation Handbook

Aditya Udai

2026

Contents

List of Figures

List of Tables

Part I

Fundamentals

Introduction to Operating Systems

“An operating system is the most important software layer between hardware and applications.”Operating Systems Principle

1.1 Chapter Overview

This chapter introduces the foundational concepts of Operating Systems (OS), one of the most important subjects in Software Development Engineer (SDE) interviews. Understanding operating systems helps engineers write efficient software, debug performance issues, reason about concurrency, and design scalable systems.

Major technology companies such as Amazon, Google, Microsoft, Meta, Uber, Netflix, Atlassian, Apple, and Bloomberg frequently ask operating system questions during interviews.

1.2 Learning Objectives

After completing this chapter, you should be able to:

- Explain what an Operating System is.
- Describe major goals of operating systems.
- Understand the evolution of operating systems.
- Compare batch, multiprogramming, and time-sharing systems.
- Understand distributed, real-time, and embedded operating systems.
- Explain kernel responsibilities.
- Differentiate user mode and kernel mode.
- Understand system calls.
- Explain the OS boot process.
- Discuss why operating systems matter for software engineers.

1.3 What is an Operating System?

1.3.1 Definition

An **Operating System (OS)** is system software that manages computer hardware and software resources and provides services to applications.

It acts as an intermediary between:

- Users
- Applications
- Hardware

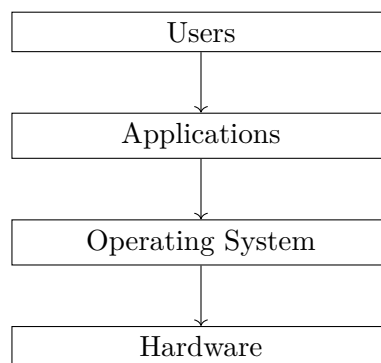


Figure 1.1: Operating System as an Interface

1.3.2 Simple Analogy

Consider a restaurant:

- Customers = Users
- Waiter = Operating System
- Kitchen = Hardware

Customers never directly communicate with the kitchen. The waiter coordinates requests and responses.

Similarly, applications do not directly access hardware. The operating system manages all interactions.

1.3.3 Examples of Operating Systems

- Linux
- Windows
- macOS

- Android
- iOS
- FreeBSD
- RTLinux
- VxWorks

1.4 Goals of an Operating System

The primary goals of an operating system are:

1.4.1 Convenience

Provide an easy environment for running applications.

1.4.2 Efficiency

Utilize hardware resources efficiently.

1.4.3 Resource Sharing

Allow multiple users and programs to share resources safely.

1.4.4 Reliability

Prevent crashes and ensure system stability.

1.4.5 Security

Protect users and resources from unauthorized access.

1.4.6 Scalability

Support growth in users, processes, and hardware.

Goal	Description
Convenience	Simplifies application development
Efficiency	Maximizes CPU and memory usage
Reliability	Ensures correct operation
Security	Protects data and resources
Scalability	Handles increasing workloads

Table 1.1: Goals of an Operating System

1.5 Evolution of Operating Systems

Operating systems evolved through several stages.

1. No Operating System
2. Batch Systems
3. Multiprogramming Systems
4. Time-Sharing Systems
5. Personal Computer Operating Systems
6. Distributed Systems
7. Mobile Operating Systems
8. Cloud and Container-Oriented Systems

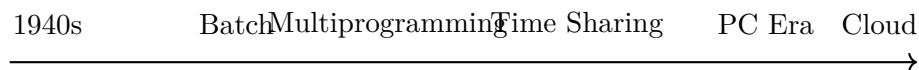


Figure 1.2: Evolution of Operating Systems

1.6 Batch Systems

1.6.1 Definition

Batch systems execute jobs in batches without user interaction.

1.6.2 Working

1. User submits jobs.
2. Jobs are grouped.
3. OS executes them sequentially.

1.6.3 Advantages

- Simple implementation
- Better CPU utilization than manual execution

1.6.4 Disadvantages

- No interaction
- Long waiting time

1.6.5 Example

Early IBM mainframe systems.

1.7 Multiprogramming Systems

1.7.1 Definition

Multiple programs reside in memory simultaneously.

When one process waits for I/O, another process uses the CPU.

1.7.2 Goal

Increase CPU utilization.

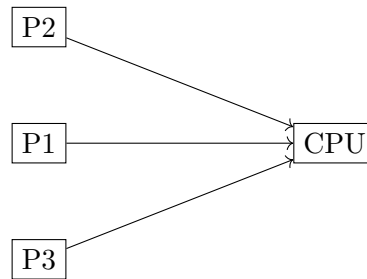


Figure 1.3: Multiprogramming Concept

1.7.3 Example

Early UNIX systems.

1.8 Time Sharing Systems

1.8.1 Definition

Time-sharing systems allow multiple users to interact with a computer simultaneously.

1.8.2 How It Works

CPU time is divided into small slices called **time quanta**.

1.8.3 Advantages

- Interactive computing

- Improved user experience
- Fair CPU allocation

1.8.4 Examples

- UNIX
- Linux
- Windows

1.9 Distributed Operating Systems

1.9.1 Definition

A distributed OS manages multiple computers and presents them as a single system.

1.9.2 Goals

- Resource sharing
- Fault tolerance
- Scalability

1.9.3 Examples

- Amoeba
- Plan 9
- Modern cloud infrastructures

1.10 Real-Time Operating Systems

1.10.1 Definition

A Real-Time Operating System (RTOS) guarantees response within strict timing constraints.

1.10.2 Types

1. Hard Real-Time
2. Soft Real-Time

Hard RTOS	Soft RTOS
Missing deadline unacceptable	Missing deadline tolerated
Aircraft systems	Video streaming

Table 1.2: Hard vs Soft RTOS

1.10.3 Examples

- VxWorks
- QNX
- RTLinux

1.11 Embedded Operating Systems

1.11.1 Definition

Embedded operating systems run on dedicated devices.

1.11.2 Characteristics

- Low memory footprint
- High reliability
- Energy efficient

1.11.3 Examples

- FreeRTOS
- Embedded Linux
- Zephyr

1.12 Components of an Operating System

Major OS components include:

- Process Management
- Memory Management
- File System
- I/O Management
- Security
- Networking

- Device Drivers

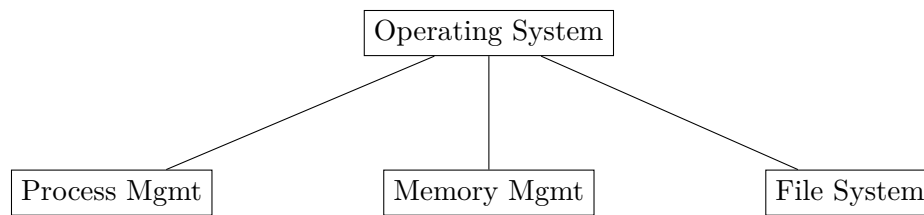


Figure 1.4: Major OS Components

1.13 Kernel Overview

1.13.1 Definition

The kernel is the core component of an operating system.

It manages:

- CPU
- Memory
- Devices
- Processes

1.13.2 Kernel Responsibilities

- Process scheduling
- Memory management
- Interrupt handling
- Device communication
- Security enforcement

1.13.3 Types of Kernels

Monolithic	Services inside kernel space
Microkernel	Minimal kernel functionality
Hybrid	Combination approach

Table 1.3: Kernel Types

1.13.4 Examples

Linux uses a monolithic kernel.

Windows uses a hybrid kernel.

1.14 User Mode vs Kernel Mode

Feature	User Mode	Kernel Mode
Privilege	Restricted	Full Access
Hardware Access	No	Yes
Crash Impact	Process Only	Entire System

Table 1.4: User Mode vs Kernel Mode

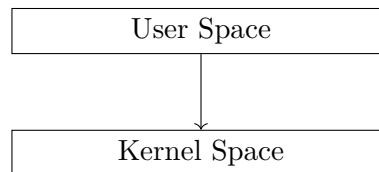


Figure 1.5: Mode Transition

1.15 System Calls

System calls provide an interface between applications and the operating system.

1.15.1 Examples

Linux:

```

read()
write()
fork()
exec()
open()
close()
  
```

Windows:

```

CreateProcess()
ReadFile()
WriteFile()
  
```

1.15.2 Flow

Application → System Call → Kernel

1.16 Boot Process Overview

1. Power On
2. BIOS/UEFI Initialization
3. Bootloader Execution
4. Kernel Loading
5. Hardware Initialization
6. System Services Start
7. User Login



Figure 1.6: Boot Sequence

1.17 Operating System Services

Operating systems provide:

- Program execution
- I/O operations
- File manipulation
- Communication
- Error detection
- Resource allocation
- Security

1.18 Why Operating Systems Matter for SDEs

1.18.1 Interview Perspective

Companies ask OS questions because engineers must understand:

- Concurrency
- Threading
- Memory usage
- Scheduling
- Performance bottlenecks
- Scalability

1.18.2 Real-World Examples

- High CPU usage debugging
- Memory leaks
- Deadlock investigation
- Container scheduling
- Distributed systems design

1.18.3 Interview Tip

Always connect operating system concepts with:

- Performance
- Scalability
- Reliability
- Security

1.19 Key Takeaways

- Operating systems manage hardware resources.
- Kernels are the core of operating systems.
- System calls enable user-kernel interaction.
- Multiprogramming improves CPU utilization.
- Time sharing enables interactivity.
- RTOS systems focus on deadlines.
- Distributed systems provide scalability.
- OS knowledge is critical for SDE interviews.

1.20 15 Interview Questions with Answers

1. What is an operating system?
Answer: System software managing hardware and providing services.
2. What are the main goals of an OS?
Answer: Convenience, efficiency, security, reliability.
3. What is a kernel?
Answer: Core component managing resources.
4. What is multiprogramming?
Answer: Multiple programs in memory simultaneously.

5. What is time sharing?
Answer: CPU sharing through time slices.
6. Difference between user and kernel mode?
Answer: Privilege level and hardware access.
7. What is a system call?
Answer: Interface to request kernel services.
8. What is a distributed OS?
Answer: OS managing multiple machines as one.
9. What is RTOS?
Answer: OS with deterministic response times.
10. What is a bootloader?
Answer: Program loading the kernel.
11. Why are interrupts important?
Answer: Enable event-driven processing.
12. Linux kernel type?
Answer: Monolithic.
13. Windows kernel type?
Answer: Hybrid.
14. Why does OS scheduling matter?
Answer: Efficient CPU allocation.
15. Why do SDE interviews ask OS questions?
Answer: To evaluate systems understanding.

1.21 15 MCQs with Answers

1. OS primarily acts as:
 - (a) Compiler
 - (b) Interface between hardware and applications
 - (c) Database
 - (d) BrowserAnswer: B
2. Kernel runs in: A) User mode B) Kernel mode Answer: B
3. Linux kernel is: A) Microkernel B) Hybrid C) Monolithic D) Exokernel Answer: C
4. Multiprogramming improves: Answer: CPU Utilization
5. RTOS stands for: Answer: Real-Time Operating System
6. System calls are used for: Answer: Accessing OS services
7. Time-sharing systems use: Answer: Time quanta
8. BIOS executes before: Answer: Kernel
9. Embedded OS targets: Answer: Dedicated devices

10. User mode has: Answer: Restricted privileges
11. Kernel mode has: Answer: Full privileges
12. Process management is handled by: Answer: OS
13. Example RTOS: Answer: VxWorks
14. Example desktop OS: Answer: Windows
15. Example distributed OS: Answer: Amoeba

1.22 Revision Notes

- OS = Resource Manager + Abstraction Layer.
- Kernel is the heart of the OS.
- User mode is restricted.
- Kernel mode has full access.
- System calls cross privilege boundaries.
- Multiprogramming increases CPU utilization.
- Time sharing improves responsiveness.
- RTOS guarantees timing constraints.
- Embedded OS focuses on resource efficiency.
- Distributed OS improves scalability.

1.23 Cheat Sheet

Operating System	Resource manager and abstraction layer
Kernel	Core OS component
System Call	Interface to kernel services
Multiprogramming	Multiple programs in memory
Time Sharing	CPU divided into quanta
RTOS	Deadline-oriented OS
Distributed OS	Multiple machines appear as one
User Mode	Restricted execution
Kernel Mode	Privileged execution
Boot Process	BIOS → Bootloader → Kernel

Computer System Architecture

2.1 Introduction

Computer System Architecture describes the organization and interaction of hardware components that execute programs and support operating systems. Understanding architecture is fundamental for operating systems because the OS directly manages processors, memory, devices, interrupts, and protection mechanisms.

From an interview perspective, computer architecture concepts frequently appear in discussions about processes, memory management, concurrency, virtualization, performance optimization, and system design.

2.2 Computer System Overview

A computer system consists of hardware and software working together to execute instructions and manage resources.

2.2.1 Major Components

- Central Processing Unit (CPU)
- Main Memory (RAM)
- Secondary Storage
- Input Devices
- Output Devices
- System Bus
- I/O Controllers
- Operating System

2.2.2 High-Level Architecture

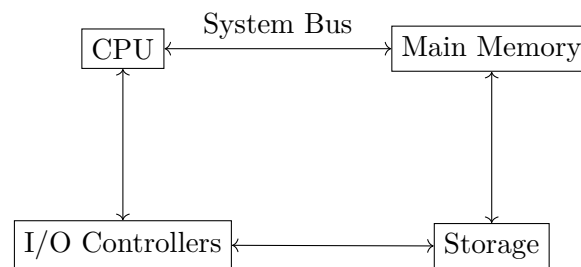


Figure 2.1: Basic Computer System Architecture

2.2.3 Von Neumann Architecture

Most modern systems follow the Von Neumann model.

Characteristics:

- Shared memory for instructions and data
- Sequential instruction execution
- Program stored in memory
- CPU fetches instructions from memory

2.3 CPU Architecture

The CPU executes instructions and coordinates system activities.

2.3.1 CPU Components

- Arithmetic Logic Unit (ALU)
- Control Unit
- Registers
- Cache
- Instruction Decoder
- Execution Units

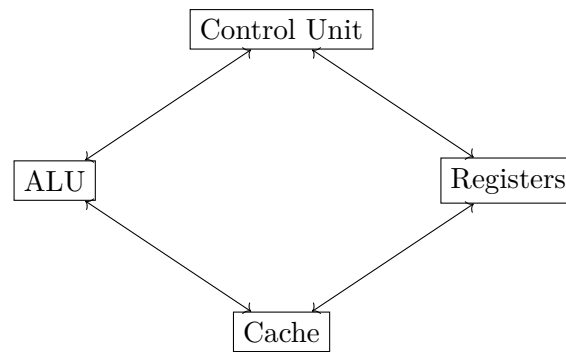


Figure 2.2: CPU Internal Organization

2.4 Registers

Registers are the fastest storage locations in a computer.

2.4.1 Characteristics

- Located inside CPU
- Very low access latency
- Hold frequently used data
- Limited quantity

2.4.2 Common Registers

Register	Purpose
General Purpose Registers	Store operands and intermediate results
Program Counter	Address of next instruction
Stack Pointer	Points to stack top
Instruction Register	Current instruction
Status Register	Condition flags
Base Register	Memory relocation
Index Register	Address calculation

2.5 Program Counter

Program Counter (PC) stores the address of the next instruction to execute.

2.5.1 Operation

1. Fetch instruction from PC address
2. Execute instruction
3. Increment PC
4. Update PC for branch instructions

2.5.2 Example

PC = 1000

Instruction at 1000 executes.

PC becomes:

1004

assuming instruction length is 4 bytes.

2.6 Stack Pointer

Stack Pointer (SP) points to the top of the stack.

2.6.1 Uses

- Function calls
- Local variables
- Return addresses
- Context switching
- Interrupt handling

2.6.2 Stack Layout

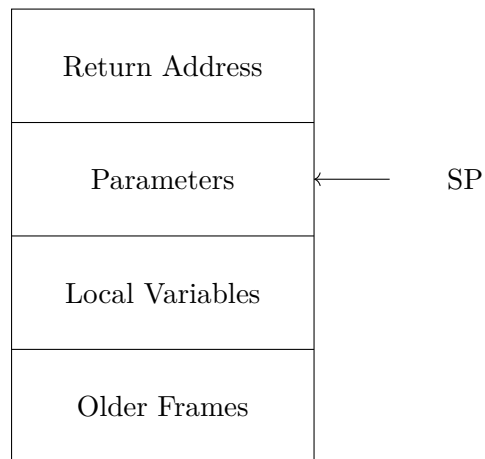


Figure 2.3: Stack Structure

2.7 Memory Hierarchy

Memory hierarchy balances cost, speed, and capacity.

2.7.1 Hierarchy Levels

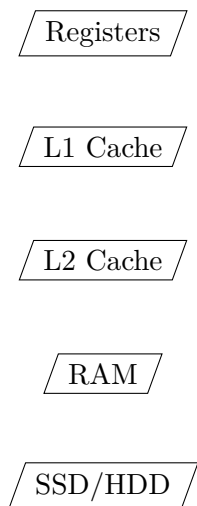


Figure 2.4: Memory Hierarchy

2.7.2 Hierarchy Comparison

Level	Speed	Cost	Capacity
Registers	Highest	Highest	Lowest
L1 Cache	Very High	High	Small

L2 Cache	High	Medium	Moderate
RAM	Moderate	Lower	Large
Disk	Slow	Lowest	Very Large

2.8 Instruction Cycle

Instruction cycle is the sequence of operations required to execute an instruction.

Stages:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

2.9 Fetch Decode Execute Cycle

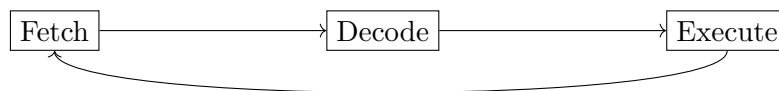


Figure 2.5: Fetch Decode Execute Cycle

2.9.1 Fetch

CPU retrieves instruction from memory.

2.9.2 Decode

Instruction decoder interprets operation.

2.9.3 Execute

ALU or execution units perform requested task.

2.10 Interrupts

Interrupts notify CPU about events requiring immediate attention.

2.10.1 Types

- Hardware Interrupts
- Software Interrupts
- Maskable Interrupts
- Non-Maskable Interrupts

2.10.2 Interrupt Handling Flow

1. Device generates interrupt
2. CPU saves context
3. Interrupt vector lookup
4. ISR executes
5. Context restored
6. Resume execution



Figure 2.6: Interrupt Processing

2.11 Trap Handling

A trap is a software-generated interrupt.

2.11.1 Examples

- System calls
- Breakpoints
- Debugging operations

2.11.2 Flow

1. User process issues trap
2. CPU switches to kernel mode
3. Kernel executes service
4. Return to user mode

2.12 Exceptions

Exceptions are unexpected conditions occurring during instruction execution.

2.12.1 Examples

- Divide by zero
- Invalid opcode
- Page fault
- Arithmetic overflow

2.12.2 Classification

Type	Description
Fault	Recoverable exception
Trap	Intentional exception
Abort	Severe unrecoverable error

2.13 DMA

Direct Memory Access allows devices to transfer data directly to memory without CPU involvement.

2.13.1 Advantages

- Reduces CPU overhead
- Improves throughput
- Enables parallelism

2.13.2 DMA Operation

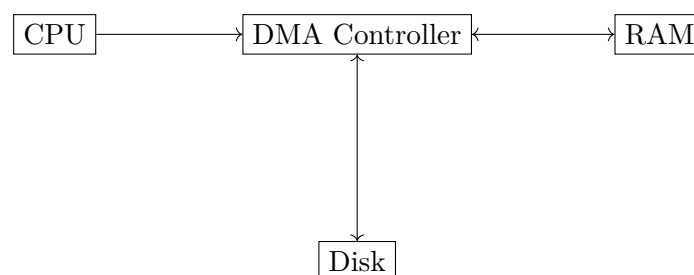


Figure 2.7: DMA Data Transfer

2.14 Cache Memory

Cache memory reduces average memory access time.

2.14.1 Why Cache Works

Based on locality principles:

- Temporal Locality
- Spatial Locality

2.14.2 Cache Levels

- L1 Cache
- L2 Cache
- L3 Cache

2.14.3 Cache Metrics

[Hit Ratio =====

Cache Hits $\frac{\text{Cache Hits}}{\text{Total Accesses}}$

2.14.4 Interview Insight

A cache hit is significantly faster than a cache miss because the CPU avoids accessing slower memory.

2.15 Multicore Systems

Modern CPUs contain multiple processing cores.

2.15.1 Benefits

- Parallel execution
- Better throughput
- Improved responsiveness
- Efficient multitasking

2.15.2 Architecture

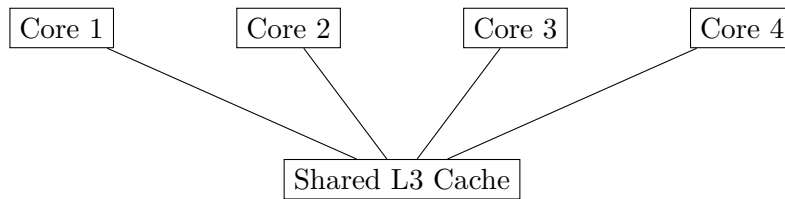


Figure 2.8: Multicore Processor

2.16 NUMA Architecture

NUMA stands for Non-Uniform Memory Access.

2.16.1 Concept

Memory access time depends on memory location relative to processor.

2.16.2 Architecture



Figure 2.9: NUMA Organization

2.16.3 Characteristics

- Local memory access is faster
- Remote memory access is slower
- Common in large servers

2.17 Hardware Protection Mechanisms

Protection mechanisms prevent unauthorized access.

2.17.1 Goals

- Isolation
- Security

- Reliability
- Resource control

2.17.2 Mechanisms

- Memory Protection
- Privilege Levels
- MMU
- Access Control Bits
- Segmentation
- Paging

2.18 User Mode and Privileged Instructions

Modern processors support multiple execution modes.

2.18.1 User Mode

- Restricted access
- Application execution
- Cannot execute privileged instructions

2.18.2 Kernel Mode

- Full hardware access
- Executes operating system code
- Handles interrupts and exceptions

2.18.3 Privileged Instructions

Examples:

- Modify page tables
- Disable interrupts
- Access device registers
- Change processor mode

2.18.4 Mode Transition

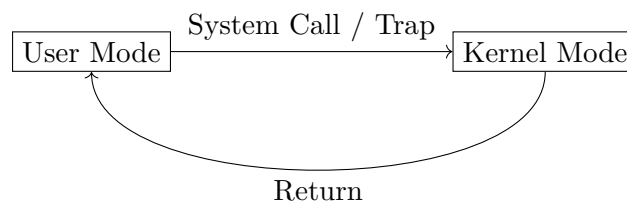


Figure 2.10: Mode Switching

2.19 Interview Explanations

2.19.1 Why Are Registers Faster Than RAM?

Registers reside inside the CPU and can be accessed without traversing memory buses.

2.19.2 Why Are Interrupts Important?

Without interrupts, CPUs would continuously poll devices, wasting processing cycles.

2.19.3 Why Is Cache Needed?

Processor speeds have increased much faster than memory speeds. Cache bridges this gap.

2.19.4 Difference Between Interrupt and Trap

Interrupts are usually external events while traps are software-generated.

2.19.5 Why Does NUMA Exist?

NUMA improves scalability in multiprocessor systems by distributing memory closer to processors.

2.20 Key Takeaways

Part II

Processes and Threads

Processes

3.1 Chapter

Overview

Processes are among the most frequently tested operating system topics in Software Development Engineer interviews. Companies such as Amazon, Google, Microsoft, Meta, Uber, Atlassian, Bloomberg, and Apple expect candidates to understand process creation, process management, context switching, scheduling, and Linux process internals.

This chapter builds a strong conceptual and interview-oriented understanding of processes.

3.2 Learning

Objectives

After completing this chapter, you should be able to:

- Define a process and explain its characteristics.
- Differentiate between a program and a process.
- Understand process memory layout.
- Explain process states and state transitions.
- Describe the Process Control Block (PCB).
- Understand context switching.
- Explain process creation and termination.
- Understand `fork()`, `exec()`, and `wait()`.
- Identify zombie and orphan processes.
- Understand daemon processes.
- Explain Linux process management.

3.3 Process

Concept

3.3.1 Definition

A **process** is a program in execution.

A process consists of:

- Program code
- Program counter
- CPU registers
- Stack
- Heap
- Open files
- Process state

3.3.2 Real-World

Analogy

Consider a recipe and a chef.

- Recipe = Program
- Chef executing recipe = Process

Multiple chefs can execute the same recipe simultaneously.

Similarly, multiple processes can execute the same program.

3.3.3 Examples

- Google Chrome
- VS Code
- MySQL Server
- Python Interpreter

Each running instance is a process.

3.4 Program

vs

Process

Feature	Program	Process
Definition	Passive entity	Active entity
Stored On	Disk	Memory
Execution	Not executing	Executing
State	Static	Dynamic
Resources	No resources allocated	Resources allocated
Example	chrome.exe	Running Chrome instance

Table 3.1: Program vs Process

3.4.1 Interview

Tip

A program becomes a process when loaded into memory and executed by the CPU.

3.5 Process

Memory

Layout

Every process has its own virtual address space.

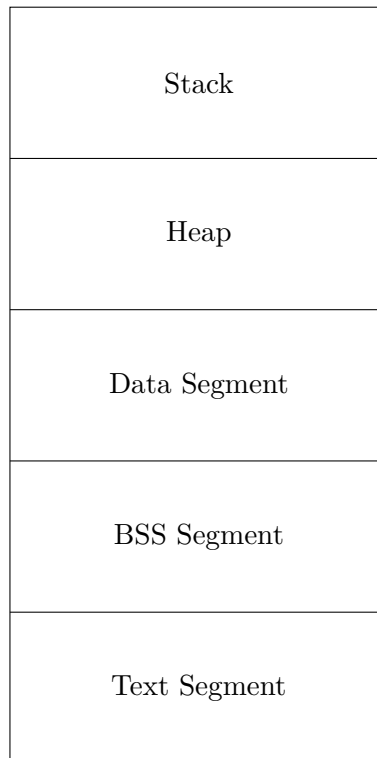


Figure 3.1: Typical Process Memory Layout

3.5.1 Text

Segment

Contains executable instructions.

3.5.2 Data

Segment

Stores initialized global and static variables.

3.5.3 BSS

Segment

Stores uninitialized global and static variables.

3.5.4 Heap

Used for dynamic memory allocation.

Examples:

`malloc()`

`calloc()`

`new`

3.5.5 Stack

Stores:

- Function calls
- Local variables
- Return addresses

3.5.6 Interview

Question

Stack grows in which direction?

Typically toward lower memory addresses.

Heap grows in which direction?

Typically toward higher memory addresses.

3.6 Process

States

3.6.1 Five-State

Model

1. New
2. Ready
3. Running
4. Waiting (Blocked)
5. Terminated

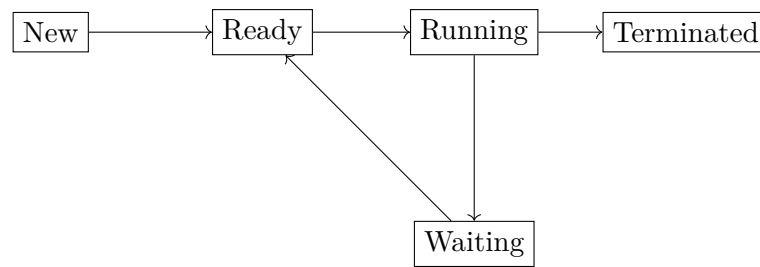


Figure 3.2: Process State Transition Diagram

3.6.2 Ready State

Waiting for CPU.

3.6.3 Running State

Currently executing.

3.6.4 Waiting State

Waiting for an event or I/O completion.

3.6.5 Terminated State

Execution completed.

3.7 Process Control Block (PCB)

3.7.1 Definition

PCB is a kernel data structure containing information about a process.

3.7.2 Contents of PCB

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Scheduling Information
- Memory Information
- Open Files

- Accounting Information

PID
State
Registers
Program Counter
Memory Info
Scheduling Info
Open Files
Accounting Info

Figure 3.3: Process Control Block

3.8 Context

Switching

3.8.1 Definition

Context switching is the process of saving the state of one process and loading the state of another.

3.8.2 Steps

1. Save current process context into PCB.
2. Select next process.
3. Load next process context.
4. Resume execution.



Figure 3.4: Context Switch

3.8.3 Interview

Insight

Context switching is pure overhead because no useful work is performed during the switch.

3.9 Process

Creation

Most operating systems allow processes to create child processes.

3.9.1 Linux

Model

Parent process creates child process using:

```
fork()
```

3.9.2 Windows

Model

Uses:

```
CreateProcess()
```

3.10 fork()

3.10.1 Definition

`fork()` creates a new process by duplicating the calling process.

3.10.2 Return

Values

Return Value	Meaning
0	Child Process
>0	Parent Process
-1	Error

Table 3.2: `fork()` Return Values

3.10.3 Example

```

1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main()
5  {
6      pid_t pid = fork();
7
8      if(pid == 0)
9          printf("Child Process\n");
10     else

```

```
11     printf("Parent Process\n");
12
13     return 0;
14 }
```

3.10.4 Interview

Insight

After fork():

- Parent and child have separate address spaces.
- Modern systems use Copy-On-Write (COW).

3.11 exec()

3.11.1 Definition

exec() replaces the current process image with a new program.

3.11.2 Common

Variants

- execl()
- execlp()
- execv()
- execvp()

3.11.3 Example

```
1  #include <unistd.h>
2
3  int main()
4  {
5      execl("/bin/ls", "ls", "-l", NULL);
6      return 0;
7  }
```

3.11.4 Important

Point

exec() does not create a new process.

It replaces the current process image.

3.12 wait()

3.12.1 Definition

wait() causes the parent process to wait for child termination.

3.12.2 Example

```
1 #include <sys/wait.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     if(fork()==0)
7         return 0;
8
9     wait(NULL);
10 }
```

3.12.3 Purpose

Prevents zombie processes.

3.13 Process

Scheduling

Overview

The scheduler selects a process from the ready queue.

Goals:

- High CPU utilization
- Fairness
- Low response time
- High throughput

Common scheduling algorithms:

- FCFS
- SJF
- Priority Scheduling
- Round Robin
- Multilevel Queue

3.14 Process

Termination

A process may terminate due to:

- Normal completion
- Error
- Fatal exception
- Parent termination
- Explicit kill signal

Linux commands:

```
kill PID
kill -9 PID
pkill process_name
```

3.15 Zombie

Processes

3.15.1 Definition

A zombie process is a terminated process whose PCB still exists.

3.15.2 Reason

Parent has not collected exit status.

3.15.3 Characteristics

- Process finished execution.
- Entry remains in process table.
- Consumes PID slot.

3.15.4 Fix

Parent must call:

```
wait()
waitpid()
```


3.15.5 Linux**Example**

```
ps aux | grep Z
```

3.16 Orphan**Processes****3.16.1 Definition**

An orphan process is a child process whose parent terminates before it.

3.16.2 What**Happens?**

The orphan is adopted by:

```
init (PID 1)
```

or

```
systemd
```

3.16.3 Interview**Question****Zombie vs Orphan?**

Zombie: Parent alive, child terminated.

Orphan: Parent terminated, child alive.

3.17 Daemon**Processes****3.17.1 Definition**

A daemon is a background process providing system services.

3.17.2 Characteristics

- No terminal
- Long-running
- Starts during boot

Examples:

- sshd
- cron
- systemd
- httpd

Linux command:

```
ps -ef
```

3.18 Process

Hierarchies

Processes form a tree structure.

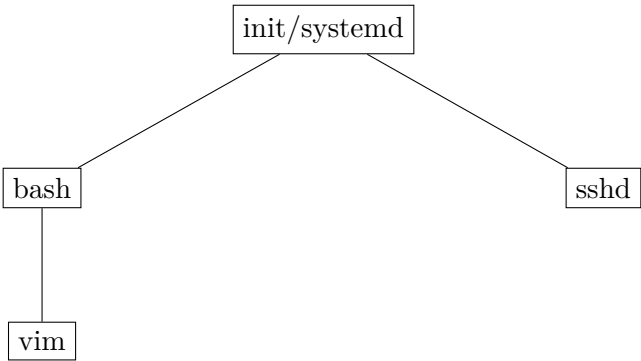


Figure 3.5: Process Hierarchy

3.19 Linux

Process

Management

3.19.1 Useful

Commands

Command	Purpose
ps	Display processes
top	Process monitoring
htop	Interactive monitoring
pstree	Process tree
kill	Terminate process
nice	Set priority
renice	Modify priority

Table 3.3: Linux Process Commands

3.19.2 Examples

View process tree:

`ps tree`

Monitor processes:

`top`

Kill process:

`kill -9 1234`

3.20 Interview

Insights

Frequently asked topics:

- `fork()` vs `exec()`
- Zombie vs Orphan
- PCB contents
- Context switching overhead
- Copy-On-Write
- Process states
- Linux process commands
- Process hierarchy

Golden Rule:

If asked about process creation in Linux:

`fork()` → `exec()` → `wait()`

3.21 Key

Takeaways

- Process is a program in execution.
- PCB stores process information.
- Context switching saves and restores process state.
- `fork()` creates a child process.
- `exec()` replaces process image.
- `wait()` synchronizes parent and child.
- Zombies are terminated processes awaiting cleanup.
- Orphans are adopted by `init`/`systemd`.
- Daemons provide background services.
- Linux provides powerful process management tools.

3.22 20 Interview Questions with Answers

1. What is a process?
Answer: Program in execution.
2. Program vs Process?
Answer: Program is passive; process is active.
3. What is PCB?
Answer: Kernel data structure storing process information.
4. What is PID?
Answer: Process Identifier.
5. What is context switching?
Answer: Saving one process state and restoring another.
6. Why is context switching expensive?
Answer: It performs no useful work.
7. What does fork() do?
Answer: Creates a child process.
8. fork() return value in child?
Answer: 0.
9. fork() return value in parent?
Answer: Child PID.
10. What does exec() do?
Answer: Replaces process image.
11. Does exec() create a process?
Answer: No.
12. Purpose of wait()?
Answer: Waits for child termination.
13. What is a zombie process?
Answer: Terminated process awaiting cleanup.
14. What is an orphan process?
Answer: Child whose parent terminated.
15. Who adopts orphan processes?
Answer: init/systemd.
16. What is a daemon?
Answer: Background service process.
17. What is Copy-On-Write?
Answer: Memory copied only upon modification.
18. What is process scheduling?
Answer: Selecting next process for CPU.
19. What is a ready queue?
Answer: Queue of runnable processes.

20. What Linux command shows processes?

Answer: ps.

3.23 20 MCQs with Answers

1. Process is: A) Hardware B) Program in execution C) Compiler D) Kernel

Answer: B

2. PCB stands for: Answer: Process Control Block

3. fork() creates: Answer: Child Process

4. exec() creates a new process. Answer: False

5. wait() prevents: Answer: Zombie Processes

6. Process state waiting means: Answer: Waiting for event/I/O

7. Linux process ID is: Answer: PID

8. Stack stores: Answer: Function calls

9. Heap stores: Answer: Dynamic allocations

10. Zombie process is: Answer: Terminated but not reaped

11. Orphan process is: Answer: Parent terminated

12. Daemon process runs: Answer: Background

13. Scheduler selects from: Answer: Ready Queue

14. Context switch stores: Answer: CPU State

15. Linux tree command: Answer: pstree

16. kill -9 sends: Answer: SIGKILL

17. Child PID returned to: Answer: Parent

18. Program becomes process when: Answer: Executed

19. Copy-On-Write improves: Answer: Efficiency

20. Running process uses: Answer: CPU

3.24 Revision

Notes

- Process = Program + Execution State.
- PCB stores process metadata.
- fork() duplicates process.
- exec() replaces process image.
- wait() synchronizes parent and child.
- Context switching causes overhead.
- Zombie = dead child not reaped.
- Orphan = living child without parent.
- Daemon = background service.

- Linux process tree begins at init/systemd.

3.25 Cheat

Sheet

Process	Program in execution
PCB	Process metadata
fork()	Create child process
exec()	Replace process image
wait()	Wait for child
PID	Process identifier
Zombie	Dead child not cleaned
Orphan	Child without parent
Daemon	Background process
Ready State	Waiting for CPU
Running State	Using CPU
Waiting State	Waiting for event
Context Switch	Save + Restore CPU state
pstree	Show hierarchy
top	Monitor processes
kill -9	Force termination

Threads

4.1 Motivation for Threads

Threads enable concurrent execution within a process. They improve responsiveness, resource sharing, scalability, and CPU utilization.

Benefits:

- Improved responsiveness
- Better resource sharing
- Lower overhead than processes
- Efficient multicore utilization
- Increased throughput

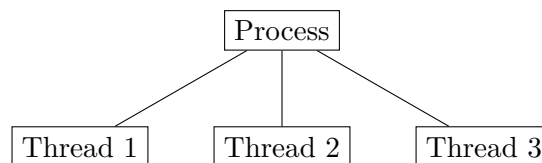


Figure 4.1: Multiple Threads Inside a Process

4.2 What is a Thread

A thread is the smallest unit of CPU execution.

Each thread contains:

- Program Counter
- Register Set
- Stack
- Thread State

Shared among threads:

- Code Segment

- Heap
- Open Files
- Global Variables

4.3 Process

vs

Thread

Feature	Process	Thread
Address Space	Separate	Shared
Creation Cost	High	Low
Context Switch	Expensive	Cheaper
Communication	IPC Needed	Shared Memory
Isolation	Strong	Weak
Failure Impact	Isolated	Can Affect Process

4.4 Thread

States

Typical thread states:

- New
- Ready
- Running
- Waiting
- Terminated

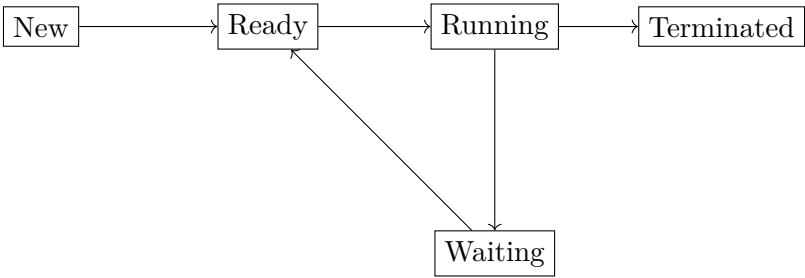


Figure 4.2: Thread State Diagram

4.5 User

Threads

User threads are managed entirely in user space.

Advantages:

- Fast creation

- Fast switching
- No kernel involvement

Disadvantages:

- Blocking system call blocks all threads
- Limited multicore support

4.6 Kernel

Threads

Kernel threads are managed by the operating system.

Advantages:

- True parallelism
- Better scheduling
- Blocking affects only one thread

Disadvantages:

- Higher overhead
- More expensive context switches

4.7 Multithreading

Models

Models define mapping between user threads and kernel threads.

4.8 Many-to-One

Multiple user threads mapped to a single kernel thread.

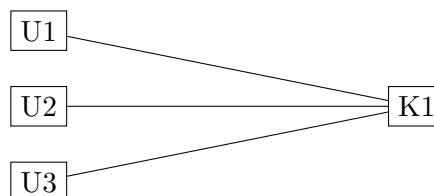


Figure 4.3: Many-to-One Model

Pros:

- Efficient

Cons:

- No parallel execution

4.9 One-to-One

Each user thread maps to one kernel thread.

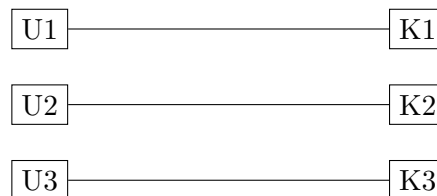


Figure 4.4: One-to-One Model

4.10 Many-to-Many

Many user threads mapped to many kernel threads.

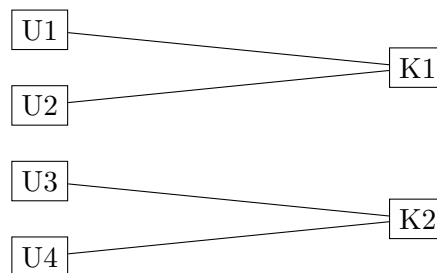


Figure 4.5: Many-to-Many Model

4.11 Thread

Libraries

Popular thread libraries:

- POSIX Threads (Pthreads)
- Java Threads
- Python threading module

4.12 POSIX

Threads

4.12.1 Pthreads

Example

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 void* work(void* arg) {
5     printf("Hello Thread\n");
6     return NULL;
7 }
8
9 int main() {
10     pthread_t t;
11     pthread_create(&t, NULL, work, NULL);
12     pthread_join(t, NULL);
13     return 0;
14 }
```

4.13 Java

Threads

Java provides Thread class and Runnable interface.

4.13.1 Java

Example

```
1 class Worker extends Thread {
2     public void run() {
3         System.out.println("Running");
4     }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
9         Worker t = new Worker();
10        t.start();
11    }
12 }
```

4.14 Python

Threads

Python provides the threading module.

4.14.1 Python

Example

```
1 import threading
2
```

```
3 def task():
4     print("Thread running")
5
6 t = threading.Thread(target=task)
7 t.start()
8 t.join()
```

4.15 Thread

Pools

Thread pools maintain a collection of reusable worker threads.

Advantages:

- Reduced creation overhead
- Better resource management
- Improved throughput
- Controlled concurrency

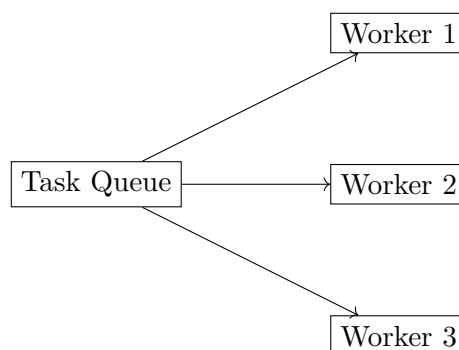


Figure 4.6: Thread Pool Architecture

4.16 Thread

Safety

Thread-safe code behaves correctly when accessed concurrently.

Techniques:

- Mutexes
- Semaphores
- Read-Write Locks
- Atomic Operations
- Immutability

4.17 Thread Local Storage

Thread Local Storage (TLS) provides thread-specific variables.

Characteristics:

- Private to each thread
- Prevents race conditions
- Useful for caches and contexts

4.18 Multicore Execution

Multiple threads can execute simultaneously on different CPU cores.



Figure 4.7: Parallel Execution on Multiple Cores

4.19 Interview Notes

- Threads share address space; processes do not.
- Context switching between threads is cheaper.
- Thread pools improve scalability.
- Kernel threads enable true parallelism.
- Thread safety is critical in concurrent systems.
- TLS provides per-thread data storage.

4.20 Key Takeaways

CPU Scheduling

5.1 Chapter

Overview

CPU Scheduling is one of the most important Operating Systems topics for Software Development Engineer interviews. It combines theoretical concepts with numerical problem-solving and is frequently asked by Amazon, Google, Microsoft, Meta, Uber, Atlassian, Apple, Bloomberg, and many other technology companies.

This chapter covers scheduling fundamentals, scheduling metrics, major scheduling algorithms, numerical techniques, interview tricks, and quick-solving strategies.

5.2 Learning

Objectives

After completing this chapter, you should be able to:

- Understand why CPU scheduling is necessary.
- Calculate scheduling performance metrics.
- Differentiate preemptive and non-preemptive scheduling.
- Solve scheduling numerals efficiently.
- Analyze FCFS, SJF, SRTF, Priority, Round Robin, MLQ, and MLFQ.
- Explain starvation and aging.
- Compare scheduling algorithms.
- Answer interview questions confidently.

5.3 CPU

Scheduling

Basics

5.3.1 Definition

CPU Scheduling is the process of selecting a process from the ready queue and allocating the CPU to it.

5.3.2 Why Scheduling is Needed

At any instant:

- Multiple processes may be ready.
- Only one process can use a CPU core.
- OS must decide who gets CPU access.

5.3.3 Scheduling Cycle

Ready Queue → Scheduler → CPU Execution

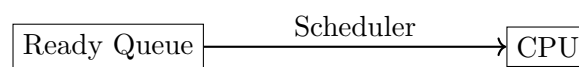


Figure 5.1: Basic CPU Scheduling

5.4 Scheduling Goals

A good scheduler should:

- Maximize CPU utilization.
- Maximize throughput.
- Minimize waiting time.
- Minimize turnaround time.
- Minimize response time.
- Ensure fairness.
- Avoid starvation.

5.5 Throughput

5.5.1 Definition

Number of processes completed per unit time.

$$Throughput = \frac{\text{Number of completed processes}}{\text{Time}}$$

5.5.2 Example

50 processes completed in 10 seconds.

$$Throughput = \frac{50}{10} = 5$$

processes/second.

5.6 Turnaround

Time

5.6.1 Definition

Total time spent by a process in the system.

$$TAT = Completion - Arrival$$

5.6.2 Example

$$Arrival = 2$$

$$Completion = 15$$

$$TAT = 15 - 2 = 13$$

5.7 Waiting

Time

5.7.1 Definition

Time spent waiting in the ready queue.

$$WT = TAT - Burst$$

5.7.2 Example

$$TAT = 20$$

$$Burst = 8$$

$$WT = 20 - 8 = 12$$

5.8 Response Time

5.8.1 Definition

Time from arrival until first CPU allocation.

$$RT = First\ Start - Arrival$$

5.8.2 Importance

Critical for interactive systems.

5.9 CPU Utilization

5.9.1 Definition

Percentage of time CPU remains busy.

$$CPU\ Utilization = \frac{CPU\ Busy\ Time}{Total\ Time} \times 100$$

5.9.2 Goal

Keep CPU utilization as high as possible.

5.10 Preemptive Scheduling

5.10.1 Definition

OS can forcibly take CPU away from a running process.

5.10.2 Examples

- Round Robin
- SRTF
- Preemptive Priority

5.10.3 Advantages

- Better responsiveness
- Fair CPU allocation

5.10.4 Disadvantages

- More context switches
- Higher overhead

5.11 Non-Preemptive Scheduling

5.11.1 Definition

Once CPU is allocated, process keeps CPU until completion or blocking.

5.11.2 Examples

- FCFS
- SJF
- Non-preemptive Priority

5.11.3 Advantages

- Simpler
- Less overhead

5.11.4 Disadvantages

- Poor responsiveness
- Convoy effect

5.12 First Come First Serve (FCFS)

5.12.1 Definition

Processes are executed in order of arrival.

5.12.2 Working

Queue behaves like FIFO.

5.12.3 Advantages

- Simple
- Easy implementation

5.12.4 Disadvantages

- Convoy effect
- Poor average waiting time

5.12.5 Complexity

$$O(1)$$

5.12.6 Example

Process	Burst
P1	5
P2	3
P3	2

Gantt Chart:

0 P1 5 P2 8 P3 10

Waiting Times:

$$P1 = 0$$

$$P2 = 5$$

$$P3 = 8$$

Average WT:

$$\frac{0 + 5 + 8}{3} = 4.33$$

5.13 Shortest Job First (SJF)

5.13.1 Definition

Shortest burst process executes first.

5.13.2 Working

Select minimum burst time.

5.13.3 Advantages

- Minimum average waiting time

5.13.4 Disadvantages

- Burst prediction difficult
- Starvation possible

5.13.5 Complexity

$$O(n)$$

5.13.6 Example

Processes:

$$P1 = 6, P2 = 2, P3 = 8, P4 = 3$$

Order:

$$P2 \rightarrow P4 \rightarrow P1 \rightarrow P3$$

Gantt Chart:

0 2 5 11 19

Average WT:

$$\frac{0 + 2 + 5 + 11}{4} = 4.5$$

5.14 Shortest Remaining Time First (SRTF)

5.14.1 Definition

Preemptive version of SJF.

5.14.2 Working

Always execute process with smallest remaining burst.

5.14.3 Advantages

- Optimal average waiting time

5.14.4 Disadvantages

- High context switching
- Starvation possible

5.14.5 Complexity

$$O(n)$$

5.14.6 Example

Arrival:

$$P1 = (0, 8)$$

$$P2 = (1, 4)$$

At time 1:

P2 preempts P1.

Gantt:

P1 | P2 | P1

5.15 Priority

Scheduling

5.15.1 Definition

Highest priority process executes first.

5.15.2 Working

Priority determines scheduling order.

5.15.3 Advantages

- Important tasks run first

5.15.4 Disadvantages

- Starvation

5.15.5 Complexity

$$O(n)$$

5.15.6 Example

Process	Priority
P1	3
P2	1
P3	2

Order:

$$P2 \rightarrow P3 \rightarrow P1$$

5.16 Round Robin Scheduling

5.16.1 Definition

Processes receive CPU for a fixed time quantum.

5.16.2 Working

After quantum expires:

- Process preempted
- Moved to queue end

5.16.3 Advantages

- Fairness
- Good response time

5.16.4 Disadvantages

- High context switching
- Quantum selection critical

5.16.5 Complexity

$$O(1)$$

5.16.6 Example

$$\text{Quantum} = 2$$

$$P1 = 5$$

$$P2 = 4$$

Gantt:

P1 P2 P1 P2 P1

5.17 Multilevel Queue Scheduling

5.17.1 Definition

Ready queue divided into multiple fixed queues.

Example:

- System Processes
- Interactive Processes
- Batch Processes

5.17.2 Working

Each queue has its own scheduler.

5.17.3 Advantages

- Separation of workloads

5.17.4 Disadvantages

- Inflexible
- Starvation possible

5.17.5 Complexity

Depends on queue structure.

5.18 Multilevel Feedback Queue (MLFQ)

5.18.1 Definition

Processes can move between queues.

5.18.2 Working

- New jobs start at top queue.
- CPU-intensive jobs move downward.
- Interactive jobs remain near top.

5.18.3 Advantages

- Adaptive
- Excellent responsiveness

5.18.4 Disadvantages

- Complex implementation

5.18.5 Complexity

Implementation dependent.

5.18.6 Example

$Q1 : RR(4)$

$Q2 : RR(8)$

$Q3 : FCFS$

5.19 Dispatcher

5.19.1 Definition

Dispatcher transfers CPU control to selected process.

Responsibilities:

- Context switching
- User mode transition
- Jump to process location

5.19.2 Dispatch**Latency**

Time taken by dispatcher to stop one process and start another.

5.20 Context**Switch****Impact**

Context switching introduces overhead.

$$Useful\ Work = Total\ Time - Context\ Switch\ Time$$

Excessive context switching reduces:

- Throughput
- CPU efficiency

5.21 Starvation**5.21.1 Definition**

A process waits indefinitely because other processes continuously receive CPU.

5.21.2 Common**In**

- Priority Scheduling
- SJF
- SRTF

5.22 Aging**5.22.1 Definition**

Gradually increasing process priority over time.

5.22.2 Purpose

Prevents starvation.

5.22.3 Example

Priority increases every 5 seconds.

5.23 Comparative

Summary

Algorithm	Preemptive	Starvation	Response Time
FCFS	No	No	Poor
SJF	No	Yes	Good
SRTF	Yes	Yes	Excellent
Priority	Both	Yes	Good
RR	Yes	No	Excellent
MLQ	Usually	Possible	Good
MLFQ	Yes	Rare	Excellent

Table 5.1: Scheduling Algorithm Comparison

5.24 Common

Interview

Traps

1. Forgetting arrival times.
2. Confusing turnaround and waiting time.
3. Using wrong quantum in RR.
4. Ignoring preemption in SRTF.
5. Incorrect completion times.
6. Missing starvation discussion.
7. Not mentioning aging.
8. Forgetting context switch overhead.

5.25 How to Solve Scheduling Numericals Quickly

5.25.1 Step 1

Create process table.

5.25.2 Step 2

Draw Gantt chart.

5.25.3 Step 3

Find completion times.

5.25.4 Step**4**

Calculate:

$$TAT = CT - AT$$

$$WT = TAT - BT$$

$$RT = FirstStart - AT$$

5.25.5 Step**5**

Compute averages.

5.25.6 Golden**Interview****Formula**

$$WT = TAT - BT$$

This formula solves most scheduling problems quickly.

5.26 Key**Takeaways**

- Scheduling selects the next CPU process.
- FCFS is simple but suffers convoy effect.
- SJF minimizes average waiting time.
- SRTF is preemptive SJF.
- Priority scheduling can starve low-priority jobs.
- Aging prevents starvation.
- Round Robin provides fairness.
- MLFQ is widely used in modern systems.
- Dispatcher performs context switches.
- Numerical problems require accurate Gantt charts.

5.27 25 Interview Questions with Answers

1. What is CPU scheduling?
Answer: Selecting a process for CPU execution.
2. What is throughput?
Answer: Completed processes per unit time.
3. Define turnaround time.
Answer: Completion time minus arrival time.
4. Define waiting time.
Answer: Time spent in ready queue.
5. Define response time.
Answer: First CPU allocation delay.
6. What is CPU utilization?
Answer: Percentage CPU remains busy.
7. FCFS stands for?
Answer: First Come First Serve.
8. Major issue in FCFS?
Answer: Convoy effect.
9. Which algorithm minimizes average waiting time?
Answer: SJF.
10. What is SRTF?
Answer: Preemptive SJF.
11. What causes starvation?
Answer: Continuous postponement.
12. How is starvation prevented?
Answer: Aging.
13. What is aging?
Answer: Gradual priority increase.
14. Round Robin is?
Answer: Preemptive.
15. What is time quantum?
Answer: Fixed CPU slice.
16. Very small quantum causes?
Answer: Excessive context switching.
17. Very large quantum becomes?
Answer: FCFS.
18. Priority scheduling problem?
Answer: Starvation.
19. What is dispatcher?
Answer: CPU transfer mechanism.

20. Dispatch latency?
Answer: Dispatcher overhead time.
21. Which algorithm is best for interactive systems?
Answer: Round Robin.
22. Which algorithm adapts dynamically?
Answer: MLFQ.
23. Which algorithm uses multiple queues?
Answer: MLQ.
24. What is context switching?
Answer: Process state change.
25. Why is scheduling important?
Answer: Efficient CPU utilization.

5.28 25 MCQs with Answers

1. FCFS is: A) Preemptive B) Non-preemptive Answer: B
2. SRTF is: Answer: Preemptive SJF
3. Turnaround Time = Answer: Completion - Arrival
4. Waiting Time = Answer: TAT - Burst
5. Response Time measures: Answer: First CPU allocation delay
6. RR uses: Answer: Time Quantum
7. Starvation occurs in: Answer: Priority Scheduling
8. Aging prevents: Answer: Starvation
9. Dispatcher performs: Answer: Context Switch
10. CPU utilization should be: Answer: High
11. Throughput should be: Answer: High
12. Waiting time should be: Answer: Low
13. Round Robin is: Answer: Preemptive
14. Convoy effect occurs in: Answer: FCFS
15. SJF gives: Answer: Minimum Average Waiting Time
16. Interactive systems prefer: Answer: RR
17. MLFQ stands for: Answer: Multilevel Feedback Queue
18. Dispatch latency is: Answer: Dispatcher Time
19. Priority scheduling can cause: Answer: Starvation
20. MLFQ allows: Answer: Queue Migration
21. Small quantum causes: Answer: High Overhead
22. Large quantum resembles: Answer: FCFS
23. Scheduler selects: Answer: Next Process

24. Throughput unit: Answer: Processes/Time
25. SRTF chooses: Answer: Shortest Remaining Burst

5.29 Revision

Notes

- CPU Scheduling selects next process.
- Throughput should increase.
- Waiting time should decrease.
- $TAT = CT - AT$.
- $WT = TAT - BT$.
- $RT = \text{First Start} - AT$.
- FCFS suffers convoy effect.
- SJF minimizes average waiting time.
- SRTF is preemptive SJF.
- RR uses time quantum.
- Aging prevents starvation.
- MLFQ is adaptive.

5.30 Cheat

Sheet

TAT	$CT - AT$
WT	$TAT - BT$
RT	$\text{First Start} - AT$
FCFS	FIFO Scheduling
SJF	Smallest Burst First
SRTF	Smallest Remaining Burst First
Priority	Highest Priority First
RR	Time Quantum Scheduling
MLQ	Fixed Multiple Queues
MLFQ	Dynamic Multiple Queues
Starvation	Indefinite Waiting
Aging	Priority Increase
Dispatcher	CPU Transfer Unit
Context Switch	Save + Restore State

Part III

Synchronization

The Critical Section Problem

6.1 Introduction to Concurrency

Concurrency refers to the execution of multiple tasks during overlapping time periods. Modern operating systems support concurrency through processes, threads, multicore processors, and distributed systems.

Concurrency improves:

- CPU utilization
- Responsiveness
- Throughput
- Scalability

However, concurrency introduces synchronization challenges when multiple execution units access shared resources.

6.1.1 Concurrent Execution

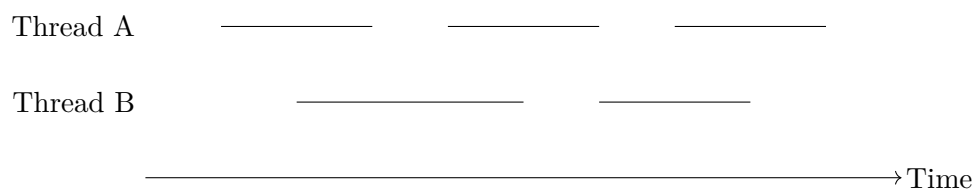


Figure 6.1: Concurrent Execution Timeline

6.2 Shared Resources

A shared resource is any resource accessible by multiple threads or processes.

Examples include:

- Shared variables
- Memory regions

- Files
- Databases
- Printers
- Network sockets

6.2.1 Shared

Counter

Example

```

1  int counter = 0;
2
3  Thread 1:
4  counter++;
5
6  Thread 2:
7  counter++;

```

At first glance the final value appears to be 2.

However concurrent execution can produce unexpected results.

6.3 Race

Conditions

A race condition occurs when the output depends on the timing or order of execution of concurrent operations.

6.3.1 Example

Assume:

counter = 5

Two threads execute:

```

1  counter++;

```

Internally:

1. Read counter
2. Increment value
3. Write back

6.3.2 Execution

Timeline

Step	Thread 1	Thread 2
1	Read 5	
2		Read 5
3	Compute 6	
4		Compute 6
5	Write 6	
6		Write 6

Final value:

6

Expected value:

7

This is a race condition.

6.4 Critical Sections

A critical section is a code segment that accesses shared resources.

Only one thread should execute a critical section at a time.

6.4.1 Structure



Figure 6.2: Critical Section Structure

6.5 The Critical Section Problem

The critical section problem asks:

How can multiple processes or threads safely share resources while preventing race conditions?

The solution must satisfy three fundamental requirements.

6.6 Requirements for a Correct Solution

6.6.1 Mutual Exclusion

At most one process can execute inside the critical section at any time.

Allowed:

$$P_1 \rightarrow CS$$

$$P_2 \rightarrow \textit{Waiting}$$

6.6.2 Progress

If no process is in the critical section and some wish to enter, selection should not be postponed indefinitely.

6.6.3 Bounded Waiting

A process should not wait forever after requesting access.

There must exist an upper bound on waiting time.

6.7 Software-Based Solutions

Before hardware synchronization primitives existed, software algorithms attempted to solve the critical section problem.

Examples:

- Peterson's Algorithm
- Bakery Algorithm

6.8 Peterson's Algorithm

Peterson's Algorithm solves the critical section problem for two processes.

6.8.1 Shared

Variables

```
1 bool flag[2];
2 int turn;
```

6.8.2 Process

i

```
1 flag[i] = true;
2 turn = j;
3
4 while(flag[j] && turn == j);
5
6 critical_section();
7
8 flag[i] = false;
```

6.8.3 Working

Principle

1. Process announces intent.
2. Gives turn to other process.
3. Waits if other process also wants entry.
4. Enters critical section safely.

6.8.4 State

Diagram

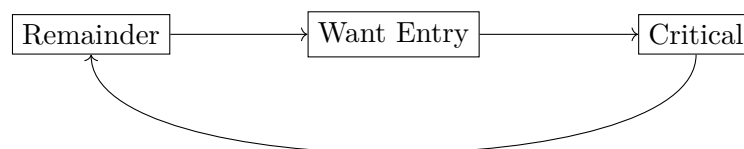


Figure 6.3: Peterson's Algorithm States

6.8.5 Interview

Note

Peterson's algorithm is important theoretically but rarely used in production systems because modern CPUs reorder memory operations.

6.9 Bakery

Algorithm

Proposed by Leslie Lamport.

Inspired by customers taking numbered tickets in a bakery.

6.9.1 Idea

1. Each process takes a number.
2. Smallest ticket enters first.
3. Ties broken using process ID.

6.9.2 Example

Process	Ticket
P1	5
P2	2
P3	7

Execution order:

$$P2 \rightarrow P1 \rightarrow P3$$

6.9.3 Advantages

- Supports multiple processes
- Guarantees fairness
- Satisfies bounded waiting

6.10 Hardware Support for Synchronization

Software-only solutions are difficult to implement correctly on modern hardware.

Processors provide atomic instructions.

Benefits:

- Faster synchronization
- Simpler implementation
- Better scalability

6.11 Atomic Operations

An atomic operation executes completely or not at all.

No interruption occurs during execution.

Examples:

- Test-and-Set
- Compare-and-Swap
- Exchange
- Atomic Increment

6.11.1 Atomicity

Illustration

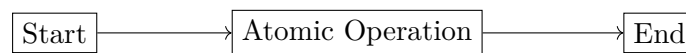


Figure 6.4: Atomic Execution

6.12 Test-and-Set

Instruction

Test-and-Set atomically reads and sets a memory location.

6.12.1 Pseudo

Code

```
1 bool TestAndSet(bool *lock)
2 {
3     bool old = *lock;
4     *lock = true;
5     return old;
6 }
```

6.12.2 Spinlock

Example

```
1 while(TestAndSet(&lock));
2
3 critical_section();
4
5 lock = false;
```

6.13 Compare-and-Swap

(CAS)

CAS compares memory content with an expected value.

If equal, replacement occurs atomically.

6.13.1 Pseudo**Code**

```
1 CAS(address, expected, newValue)
```

6.13.2 Example

```
1 int expected = 10;
2
3 CAS(&x, expected, 20);
```

If:

$$x = 10$$

then:

$$x = 20$$

Otherwise no change occurs.

6.14 Exchange**Instruction**

Exchange atomically swaps values.

6.14.1 Pseudo**Code**

```
1 Exchange(&a, &b);
```

6.14.2 Use**Cases**

- Locks
- Spinlocks
- Atomic ownership transfer

6.15 Spinlocks

A spinlock repeatedly checks a lock variable.

6.15.1 Operation

1. Attempt lock acquisition.
2. If unavailable, spin.
3. Acquire lock.
4. Execute critical section.
5. Release lock.

6.15.2 Visualization

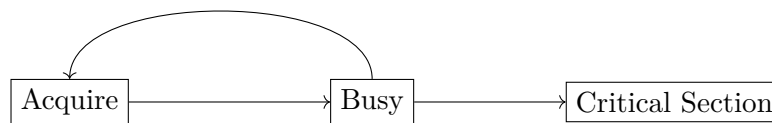


Figure 6.5: Spinlock Behavior

6.15.3 Advantages

- Low overhead
- Efficient for short waits

6.15.4 Disadvantages

- CPU wastage
- Poor for long waits

6.16 Modern CPU Challenges

Modern processors introduce complexities:

- Out-of-order execution
- Multiple cache levels
- Memory reordering
- Store buffers
- Multiple cores

Traditional algorithms may fail without memory barriers.

6.17 Memory Reordering Basics

Processors may reorder instructions to improve performance.

6.17.1 Example

Program order:

```
1 x = 1;
2 flag = true;
```

Observed order:

```
1 flag = true;
2 x = 1;
```

Another CPU may observe the reordered version.

6.17.2 Memory Illustration



Figure 6.6: Shared Memory Visibility Challenges

6.18 Real-World Race Condition Examples

6.18.1 Bank Account Update

```
1 balance = balance - 100;
```

Simultaneous withdrawals may corrupt the balance.

6.18.2 Inventory Management

Two orders simultaneously decrement stock.

Result:

Negative inventory.

6.18.3 Ticket Booking

Two users purchase the last seat concurrently.

Result:

Overbooking.

6.18.4 Web

Counter

```
1 visitors++;
```

Concurrent updates lose increments.

6.19 Interview

Explanations

6.19.1 Why is the Critical Section Problem Important?

Almost every concurrent application accesses shared resources.

Without synchronization:

- Data corruption occurs.
- Results become nondeterministic.
- Bugs become difficult to reproduce.

6.19.2 Why Isn't Peterson's Algorithm Used in Practice?

Modern hardware reorders memory operations.

The algorithm assumes strict memory ordering.

6.19.3 Why Is CAS Popular?

CAS is lock-free and supported by modern CPUs.

Many concurrent data structures use CAS internally.

6.19.4 Spinlock vs Mutex

Spinlocks waste CPU cycles while waiting.

Mutexes put threads to sleep.

Spinlocks are preferred only for short critical sections.

6.20 Common

Mistakes

- Assuming increment operations are atomic.

- Forgetting memory visibility issues.
- Holding locks too long.
- Using spinlocks for long waits.
- Ignoring fairness requirements.
- Accessing shared data without synchronization.
- Assuming multicore execution is deterministic.

6.21 Key

Takeaways

The Critical Section Problem

7.1 Introduction to Concurrency

Concurrency refers to the execution of multiple tasks during overlapping time periods. Modern operating systems support concurrency through processes, threads, multicore processors, and distributed systems.

Concurrency improves:

- CPU utilization
- Responsiveness
- Throughput
- Scalability

However, concurrency introduces synchronization challenges when multiple execution units access shared resources.

7.1.1 Concurrent Execution

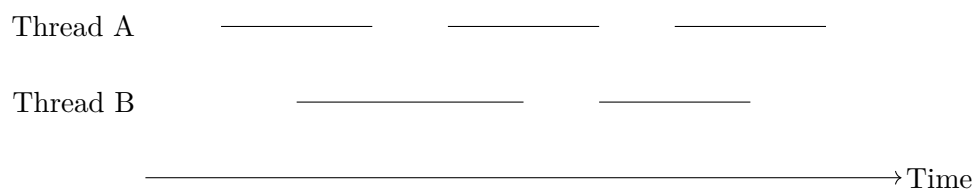


Figure 7.1: Concurrent Execution Timeline

7.2 Shared Resources

A shared resource is any resource accessible by multiple threads or processes.

Examples include:

- Shared variables
- Memory regions

- Files
- Databases
- Printers
- Network sockets

7.2.1 Shared

Counter

Example

```

1  int counter = 0;
2
3  Thread 1:
4  counter++;
5
6  Thread 2:
7  counter++;

```

At first glance the final value appears to be 2.

However concurrent execution can produce unexpected results.

7.3 Race

Conditions

A race condition occurs when the output depends on the timing or order of execution of concurrent operations.

7.3.1 Example

Assume:

counter = 5

Two threads execute:

```

1  counter++;

```

Internally:

1. Read counter
2. Increment value
3. Write back

7.3.2 Execution

Timeline

Step	Thread 1	Thread 2
1	Read 5	
2		Read 5
3	Compute 6	
4		Compute 6
5	Write 6	
6		Write 6

Final value:

6

Expected value:

7

This is a race condition.

7.4 Critical Sections

A critical section is a code segment that accesses shared resources.

Only one thread should execute a critical section at a time.

7.4.1 Structure



Figure 7.2: Critical Section Structure

7.5 The Critical Section Problem

The critical section problem asks:

How can multiple processes or threads safely share resources while preventing race conditions?

The solution must satisfy three fundamental requirements.

7.6 Requirements for a Correct Solution

7.6.1 Mutual Exclusion

At most one process can execute inside the critical section at any time.

Allowed:

$$P_1 \rightarrow CS$$

$$P_2 \rightarrow \textit{Waiting}$$

7.6.2 Progress

If no process is in the critical section and some wish to enter, selection should not be postponed indefinitely.

7.6.3 Bounded Waiting

A process should not wait forever after requesting access.

There must exist an upper bound on waiting time.

7.7 Software-Based Solutions

Before hardware synchronization primitives existed, software algorithms attempted to solve the critical section problem.

Examples:

- Peterson's Algorithm
- Bakery Algorithm

7.8 Peterson's Algorithm

Peterson's Algorithm solves the critical section problem for two processes.

7.8.1 Shared

Variables

```
1 bool flag[2];
2 int turn;
```

7.8.2 Process

i

```
1 flag[i] = true;
2 turn = j;
3
4 while(flag[j] && turn == j);
5
6 critical_section();
7
8 flag[i] = false;
```

7.8.3 Working

Principle

1. Process announces intent.
2. Gives turn to other process.
3. Waits if other process also wants entry.
4. Enters critical section safely.

7.8.4 State

Diagram

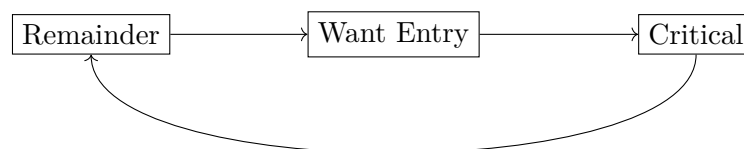


Figure 7.3: Peterson's Algorithm States

7.8.5 Interview

Note

Peterson's algorithm is important theoretically but rarely used in production systems because modern CPUs reorder memory operations.

7.9 Bakery

Algorithm

Proposed by Leslie Lamport.

Inspired by customers taking numbered tickets in a bakery.

7.9.1 Idea

1. Each process takes a number.
2. Smallest ticket enters first.
3. Ties broken using process ID.

7.9.2 Example

Process	Ticket
P1	5
P2	2
P3	7

Execution order:

$$P2 \rightarrow P1 \rightarrow P3$$

7.9.3 Advantages

- Supports multiple processes
- Guarantees fairness
- Satisfies bounded waiting

7.10 Hardware Support for Synchronization

Software-only solutions are difficult to implement correctly on modern hardware.

Processors provide atomic instructions.

Benefits:

- Faster synchronization
- Simpler implementation
- Better scalability

7.11 Atomic Operations

An atomic operation executes completely or not at all.

No interruption occurs during execution.

Examples:

- Test-and-Set
- Compare-and-Swap
- Exchange
- Atomic Increment

7.11.1 Atomicity

Illustration

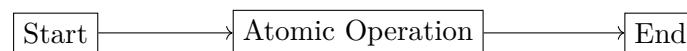


Figure 7.4: Atomic Execution

7.12 Test-and-Set

Instruction

Test-and-Set atomically reads and sets a memory location.

7.12.1 Pseudo

Code

```
1 bool TestAndSet(bool *lock)
2 {
3     bool old = *lock;
4     *lock = true;
5     return old;
6 }
```

7.12.2 Spinlock

Example

```
1 while(TestAndSet(&lock));
2
3 critical_section();
4
5 lock = false;
```

7.13 Compare-and-Swap

(CAS)

CAS compares memory content with an expected value.

If equal, replacement occurs atomically.

7.13.1 Pseudo**Code**

```
1 CAS(address, expected, newValue)
```

7.13.2 Example

```
1 int expected = 10;
2
3 CAS(&x, expected, 20);
```

If:

$x = 10$

then:

$x = 20$

Otherwise no change occurs.

7.14 Exchange**Instruction**

Exchange atomically swaps values.

7.14.1 Pseudo**Code**

```
1 Exchange(&a, &b);
```

7.14.2 Use**Cases**

- Locks
- Spinlocks
- Atomic ownership transfer

7.15 Spinlocks

A spinlock repeatedly checks a lock variable.

7.15.1 Operation

1. Attempt lock acquisition.
2. If unavailable, spin.
3. Acquire lock.
4. Execute critical section.
5. Release lock.

7.15.2 Visualization

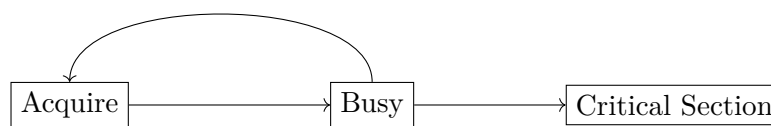


Figure 7.5: Spinlock Behavior

7.15.3 Advantages

- Low overhead
- Efficient for short waits

7.15.4 Disadvantages

- CPU wastage
- Poor for long waits

7.16 Modern CPU Challenges

Modern processors introduce complexities:

- Out-of-order execution
- Multiple cache levels
- Memory reordering
- Store buffers
- Multiple cores

Traditional algorithms may fail without memory barriers.

7.17 Memory Reordering Basics

Processors may reorder instructions to improve performance.

7.17.1 Example

Program order:

```
1 x = 1;
2 flag = true;
```

Observed order:

```
1 flag = true;
2 x = 1;
```

Another CPU may observe the reordered version.

7.17.2 Memory Illustration



Figure 7.6: Shared Memory Visibility Challenges

7.18 Real-World Race Condition Examples

7.18.1 Bank Account Update

```
1 balance = balance - 100;
```

Simultaneous withdrawals may corrupt the balance.

7.18.2 Inventory Management

Two orders simultaneously decrement stock.

Result:

Negative inventory.

7.18.3 Ticket Booking

Two users purchase the last seat concurrently.

Result:

Overbooking.

7.18.4 Web

Counter

```
1 visitors++;
```

Concurrent updates lose increments.

7.19 Interview

Explanations

7.19.1 Why is the Critical Section Problem Important?

Almost every concurrent application accesses shared resources.

Without synchronization:

- Data corruption occurs.
- Results become nondeterministic.
- Bugs become difficult to reproduce.

7.19.2 Why Isn't Peterson's Algorithm Used in Practice?

Modern hardware reorders memory operations.

The algorithm assumes strict memory ordering.

7.19.3 Why Is CAS Popular?

CAS is lock-free and supported by modern CPUs.

Many concurrent data structures use CAS internally.

7.19.4 Spinlock vs Mutex

Spinlocks waste CPU cycles while waiting.

Mutexes put threads to sleep.

Spinlocks are preferred only for short critical sections.

7.20 Common

Mistakes

- Assuming increment operations are atomic.

- Forgetting memory visibility issues.
- Holding locks too long.
- Using spinlocks for long waits.
- Ignoring fairness requirements.
- Accessing shared data without synchronization.
- Assuming multicore execution is deterministic.

7.21 Key

Takeaways

Part IV

Deadlocks

Part V

Memory Management

Part VI

File Systems

Part VII

Input/Output Systems

Part VIII

Modern Operating Systems

Part IX

Operating Systems for System Design

Part X

Interview Mastery

Bibliography
